Ejercicio 1 (1,75 ptos)

```
def lee_autocares(ruta: str) -> list[Autocar]:
  res = list()
  with open (ruta, "rt",encoding='utf-8') as f:
    it = csv.reader(f, delimiter=',')
    next(it)
    for matrícula, marca, plazas, tipo, precio, viajes in it:
      plazas = int(plazas)
      precio = int(precio)
      viajes = parsea_viajes(viajes)
      tupla = Autocar(matrícula, marca, plazas, tipo, precio, viajes)
      res.append(tupla)
  return res

def parsea_viajes(viajes_str: str) -> list[Viaje]:
  res = list()
  viajes_str = viajes_str.replace("[","").replace("]","")
  if len(viajes_str) > 0:
    for viaje in viajes_str.split("*"):
      res.append(parsea_viaje(viaje))
  return res

def parsea_viaje(viaje_str: str) -> Viaje:
    inicio, fin, kms, num_viajeros = viaje_str.split(";")
    inicio = datetime.strptime(inicio, "%d/%m/%Y %H:%M")
    fin = datetime.strptime(fin, "%d/%m/%Y %H:%M")
    kms = float(kms)
    num_viajeros = int(num_viajeros)
    return Viaje(inicio, fin, kms, num_viajeros)


## Test ========================================================================
def test_lee_autocares(autocares: list[Autocar]) -> None:
    print("="*30)
    print("Test de lectura de autocares:")
    print(f"Leídos {len(autocares)} regitros.")
    print("Mostrando los 4 primeros:")
    for autocar in autocares[:4]:
        print(autocar)

if __name__=="__main__":
    autocares = lee_autocares("data/autocares.csv")
    test_lee_autocares(autocares)
```

Ejercicio 2 (1,75 ptos)

```
# Versión con compresión
def viaje_mas_kilometros(autocares: list[Autocar], marcas: set[str]|None=None)
```

```
                        -> tuple[str, Viaje]:
    viajes_filtrados = (
        (a.matricula, v)
        for a in autocares
        if marcas is None or a.marca in marcas
        for v in a.viajes
    )
    return max(viajes_filtrados, key=lambda x: x[1].kms)

# Versión con bucles
def viaje_mas_kilometros(autocares: list[Autocar], marcas: set[str]|None=None) ->
tuple[str, Viaje]:
  res = None
  for a in autocares:
    if marcas == None or a.marca in marcas:
      for v in a.viajes:
        if res == None or res[1].kms < v.kms:
          res = (a.matricula,v)
  return res

# Otra alternativa
def viaje_mas_kilometros(autocares: list[Autocar], marcas: set[str]|None=None) ->
tuple[str, Viaje]:
    max_km = 0
    matricula_max = ""
    for autocar in autocares:
        if (marcas == None or autocar.marca in marcas) and autocar.viajes:
            viaje_max = max(autocar.viajes, key=lambda v: v.kms)
            if viaje_max.kms > max_km:
                max_km = viaje_max.kms
                matricula_max = autocar.matricula
    return matricula_max, viaje_max



## Test ========================================================================
def test_viaje_mas_kilometros(autocares: list[Autocar],
                              marcas: set[str]|None=None) -> None:
    msg ="Viaje con más kilómetros"
    if marcas is not None:
        msg = f"{msg} de las marcas {marcas}"
    print(msg+":")
    matricula, viaje = viaje_mas_kilometros(autocares)
    print(f"Matrícula: {matricula}, Viaje: {viaje}")

if __name__=="__main__":
    autocares = lee_autocares("data/autocares.csv")

    test_viaje_mas_kilometros(autocares)
```

```
    marcas = {'IRIZAR', 'SETRA', 'MERCEDES-BENZ'}
    test_viaje_mas_kilometros(autocares, marcas)
```

Ejercicio 3 (2 ptos)

```
#Usando definiciones por compresiónm
def autocar_mayor_beneficio(autocares: list[Autocar]) -> tuple[str, str, float]:
  m = max (autocares, key=beneficio_autocar)
  return (m.matricula, m.marca, beneficio_autocar(m))

# Función de beneficio por compresión
def beneficio_autocar (autocar: Autocar) -> float:
   return sum(((v.fin - v.inicio).days * autocar.precio) / v.kms
                        for v in autocar.viajes)

#Usando bucles
def autocar_mayor_beneficio2(autocares: list[Autocar]) -> tuple[str, str, float]:
  res = list()
  for a in autocares:
    res.append((a.matricula, a.marca, beneficio_autocar(a)))
  return max(res,key=lambda e:e[2])

def beneficio_autocar2 (autocar: Autocar) -> float:
    beneficio = 0
    for v in a.viajes:
      beneficio += a.precio*((v.fin.date() - v.inicio.date()).days + 1) / v.kms
    return beneficio


## Test ========================================================================
def test_autocar_mayor_beneficio(autocares: list[Autocar]) -> None:
    res = autocar_mayor_beneficio(autocares)
    print(f"El autocar con mayor benefico es: {res}")


if __name__=="__main__":
    autocares = lee_autocares("data/autocares.csv")
    test_autocar_mayor_beneficio(autocares)
```

Ejercicio 4 (2 ptos)

```
def promedio_viajeros_por_tipo (autocares: list[Autocar], fecha: date|None=None) ->
dict[str, float]:
  aux = agrupar_num_viajeros_por_tipo(autocares, fecha)
  res = dict()
  for clave,valor in aux.items():
    if len(valor) > 0:
      res[clave] = sum(valor) / len(valor)
```

```
    else:
      res[clave] = None
  return res

def agrupar_num_viajeros_por_tipo(autocares: list[Autocar], fecha:date|None=None) ->
dict[str, list[int]]:
    aux = dict()
    for a in autocares:
        if a.tipo not in aux:
            aux[a.tipo] = list()
        for v in a.viajes:
            if fecha == None or (v.inicio.date() <= fecha and fecha <=v.fin.date()):
                    aux[a.tipo].append(v.num_viajeros)
    return aux

## Test ========================================================================
def test_promedio_viajeros_por_tipo(autocares: list[Autocar], fecha: date|None=None)
                                    -> None:
    msg = "Promedio de viajeros por tipo de autocar"
    if fecha is not None:
        msg = f"{msg} el {fecha}"
    print(f"{msg}:")
    promedio = promedio_viajeros_por_tipo(autocares, fecha)
    for tipo, avg in promedio.items():
        print(f"Tipo: {tipo}, Promedio de viajeros: {avg}")
    print()

if __name__=="__main__":
    autocares = lee_autocares("data/autocares.csv")

    test_promedio_viajeros_por_tipo(autocares)
    test_promedio_viajeros_por_tipo(autocares, date(2024, 2, 10))
```

Ejercicio 5 (2,5 ptos)

```
# Creando un diccionario de agrupación y Con definiciones por compresiòn
def meses_mas_viajeros_por_tipo(autocares: list[Autocar], n: int) -> dict[str,
list[tuple[int, int]]]:
    aux = agrupar_autocares_por_tipo(autocares)

    return {tipo: meses_mas_viajeros(lista_autocares)[:n]
          for tipo, lista_autocares in aux.items()}

# Con bucles
def meses_mas_viajeros_por_tipo2(autocares: list[Autocar], n: int) -> dict[str,
list[tuple[int, int]]]:
    aux = agrupar_autocares_por_tipo(autocares)
    res = dict()
    for clave, valor in aux.items():
```

```
        res[clave] = meses_mas_viajeros(valor)[:n]
    return res

def agrupar_autocares_por_tipo(autocares: list[Autocar]) -> dict[str,
list[Autocar]]:
    res = defaultdict(list)
    for a in autocares:
        res[a.tipo].append(a)
    return res

def meses_mas_viajeros(autocares: list[Autocar]) -> list[tuple[int, int]]:
    res = defaultdict(int)
    for a in autocares:
        for v in a.viajes:
            res[v.inicio.month] += v.num_viajeros
    ## También return sorted(res.items(), key=res.get, reverse=True)
    return sorted(res.items(), key=lambda e:e[1], reverse=True)

# Creando un diccionario de diccionarios y con definiciones por compresiòn
def meses_mas_viajeros_por_tipo3(autocares: list[Autocar], n: int) -> dict[str,
list[tuple[int, int]]]:
    viajeros_por_tipo = defaultdict(lambda: defaultdict(int))
    for autocar in autocares:
        for viaje in autocar.viajes:
            mes = viaje.inicio.month
            viajeros_por_tipo[autocar.tipo][mes] += viaje.num_viajeros

    return {tipo: sorted(viajeros.items(), key=lambda x: x[1], reverse=True)[:n]
          for tipo, viajeros in viajeros_por_tipo.items()}

def test_meses_mas_viajeros_por_tipo(autocares: list[Autocar], n: int) -> None:
    meses = meses_mas_viajeros_por_tipo(autocares, n)
    for tipo, mes in meses.items():
        print(f"Tipo: {tipo}, Mes con más viajeros: {mes}")

if __name__=="__main__":
    autocares = lee_autocares("data/autocares.csv")

    test_meses_mas_viajeros_por_tipo(autocares, 4)
```